Cortex Code (CoCo) — Developer Cheatsheet

> [!IMPORTANT] Community cheatsheet — not official Snowflake documentation. For the authoritative reference, see Cortex Code CLI docs.

Table of Contents

Quick Start

Install on macOS/Linux:

```
# run in terminal
curl -sL https://sfc-repo.snowflakecomputing.com/cortex/install.sh
| bash
```

Enable shell completions (one-time setup):

```
# run in terminal
cortex completion bash > ~/.bash_completion.d/cortex   # bash
cortex completion zsh  > ~/.zsh/completions/_cortex    # zsh
cortex completion fish > ~/.config/fish/completions/cortex.fish  #
```

```
fish
exec $SHELL
```

Common launch options:

```
# run in terminal
cortex                                                          # launch in current directory
cortex -c my_connection -w /path/to/project                     # set connection and working dir
cortex --continue                                               # resume last session
cortex --resume <session_id>                                    # resume specific session by ID
cortex -p "list all Python files" --output-format stream-json   # one-off prompt (scripting)
cortex -f request.txt                                           # read prompt from file and exit
cortex -m claude-opus-4-6 -p "summarize this repo"             # specify model
```

Connections are defined in `~/.snowflake/connections.toml`. Recommended: `OAUTH_AUTHORIZATION_CODE` (browser-based OAuth — no stored password, tokens refreshed automatically):

```
[my_connection]
account                         = "<ACCOUNT_IDENTIFIER>"
user                            = "<USERNAME>"
authenticator                   = "OAUTH_AUTHORIZATION_CODE"
client_store_temporary_credential = true
role                            = "<ROLE>"
database                        = "<DATABASE>"
```

Start Here By Role

If you're a…	Focus on
DB / Data Developer	`#TABLE`, `/sql`, `cortex search table-details`, Semantic Views, SQL timeout
App Developer (Streamlit, Native Apps)	`@{file}`, `/diff`, `/worktree`, `$snowflake-apps`, `$deploy-to-spcs`
Data / ML Engineer	`/lineage`, `$machine-learning`, `$cortex-agent`, `$dynamic-tables`, fdbt
Platform / Infra	`/plan`, `/team`, `/bg`, Hooks, MCP, Scheduling, Memory

The 7 Trigger Characters

Trigger	Name	What it does	Example
`/`	Slash command	Invoke a slash command	`/sql SELECT current_user()`
`!`	Bash shell	Run a terminal command inline	`! git status`
`@`	File reference	Reference a file by path (no inject)	`@src/app.py review this`
`@{`	File inject	Inject full file contents into prompt	`@{schema.sql} write tests`
`#`	Table reference	Autocomplete table name, load schema	`#DB.PUBLIC.ORDERS summarize`
`$`	Skill invoke	Invoke a skill by name	`$semantic-view create a view for sales`
`%`	Agent mention	Mention a Cortex Agent	`%my_sales_agent what was Q1 revenue?`

> [!TIP] `@{file}` is the most useful trigger for developers: inject a config, schema, or source file into context without manually copying content.

Slash Commands You'll Actually Use

Execution & modes:

Command	What it does
`/plan`	Enter plan mode — CoCo presents a plan before making changes
`/plan-off`	Disable plan mode
`/bypass`	Auto-approve all tool calls
`/bypass-off`	Disable bypass mode
`/team`	Enable parallel teammates mode
`/bg`	Launch a background agent, keep chatting

Command	What it does
/agents	View and manage running subagents
/sandbox	Manage sandbox settings (on / off / status)

SQL & data:

Command	What it does
/sql <query>	Execute SQL inline (--limit N for row cap)
/sql-readonly	Toggle SQL write protection
/table	Open interactive table viewer
/connections	Manage Snowflake connections
/doctor	Diagnose connection issues

Code & git:

Command	What it does
/diff	Review git changes fullscreen (--staged for staged)
/worktree <create|list|switch|delete>	Manage git worktrees
/dbt	dbt operations and lineage

Skills & config: /skill /rules /hooks /mcp /settings /permissions

Utilities: /copy /share /secrets /compact /rewind /new /resume /fork /index /tgrep /tasks /rename /update

Keyboard Shortcuts That Matter

Shortcut	Action
Ctrl+P	Toggle compact/expanded mode
Ctrl+O	Open full transcript viewer
Ctrl+T	Open table viewer
Ctrl+B	View background bash processes
Ctrl+D	Open todo/task viewer
Ctrl+C	Interrupt / cancel
Shift+Tab	Cycle mode (Confirm → Plan → Bypass)
Ctrl+R	History search (reverse)
Ctrl+J	Insert newline in input (for multiline prompts)
Esc Esc	Clear entire input

Shortcut	Action
`?`	Toggle help overlay
`Ctrl+A`	Move to start of input
`Ctrl+E`	Move to end of input

Working with Your Codebase

Trigger	Example	What happens
`@file`	`@src/auth.py what does this do?`	References the path (no content injection)
`@{file}`	`@{schema.sql} write tests for this`	Injects full file contents into context
`@{f1} @{f2}`	`@{schema.sql} @{models.py} do these match?`	Inject multiple files at once

For semantic code search: `/tgrep status` to check, `/index --rebuild` after major changes.

Ask naturally — CoCo uses tgrep internally: `"find where authentication errors are handled"`

Snowflake Data Shortcuts

Reference a table to load its schema instantly:

```
#SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS how many orders this year?
```

Run SQL (use `Ctrl+J` for newlines in multiline queries):

```
/sql SELECT COUNT(*), SUM(O_TOTALPRICE) FROM
SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS
     WHERE O_ORDERDATE >= '1995-01-01'
/sql SELECT O_ORDERSTATUS, COUNT(*) AS cnt
     FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS GROUP BY 1
/sql SELECT * FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER --limit
10
```

Discover objects and check docs before writing queries:

```
# run in terminal
cortex search object "TPCH orders"
cortex search table-details
"SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS"
cortex search docs "dynamic tables target lag"
```

```
cortex lineage SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS --direction
downstream --tree
```

Or from within CoCo: `!cortex search table-details "SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.ORDERS"`

`SNOWFLAKE_SAMPLE_DATA` is in every Snowflake account (no setup). Other schemas: `TPCDS_SF10TCL` (retail), `WEATHER`.

Agent Modes

Mode	How to activate	When to use
Plan	`/plan`	Any multi-file or risky change — CoCo drafts a plan before touching anything
Bypass	`/bypass`	Auto-approve all tool calls (careful on prod)
Team	`/team`	Spawn parallel agents for large tasks
Background	`/bg`	Fire off a task, keep chatting; results arrive as a notification

`Shift+Tab` cycles through all modes: Confirm → Plan → Bypass.

Git worktrees for parallel safe work: `/worktree create <branch>` gives each agent its own branch.

Skills

Invoke with `$` prefix in the CoCo prompt:
`$cortex-agent build a Q&A agent over my warehouse`

Manage:

```
/skill              # browse installed skills
/skill list         # list all

# run in terminal
cortex skill list
cortex skill find "<query>"
cortex skill add <name>
```

Custom skills: place a `.md` file in `.cortex/skills/` (project) or `~/.snowflake/cortex/skills/` (global).

Bundled skills:

Skill	What it does
`$cortex-agent`	Build, debug, deploy Cortex Agents
`$semantic-view`	Create/manage semantic views for Cortex Analyst
`$cortex-ai-functions`	AI_CLASSIFY, AI_SENTIMENT, embeddings, OCR at scale
`$machine-learning`	Train models, use Model Registry, run ML jobs
`$dynamic-tables`	Build incremental pipelines with Dynamic Tables
`$snowpark-python`	Snowpark Python UDFs, UDAFs, stored procedures
`$iceberg`	Iceberg tables: catalog integrations, external volumes
`$snowflake-apps`	Scaffold and deploy Snowflake App Runtime apps
`$developing-with-streamlit`	Streamlit in Snowflake: theming, custom components
`$snowflake-notebooks`	Workspace notebooks: Snowpark Python + SQL cells
`$deploy-to-spcs`	Deploy containers to Snowpark Container Services
`$workload-performance-analysis`	SQL query performance: spilling, partition pruning
`$lineage`	Trace upstream/downstream column-level lineage
`$warehouse`	Analyze and right-size warehouse costs

Full list (30+): Bundled Skills

Scheduling

```
/loop                        # interactive scheduler
/loop "every day at 9am"     # natural language
/loop "0 9 * * 1-5"          # standard cron (Mon–Fri 9am)
/automation                  # schedule a recurring agent task
```

> [!NOTE] Scheduled tasks are session-scoped and expire after 3 days. Max 50 per session.

Memory Across Sessions

Enable once, then memories persist across every CoCo session.

```
# run in terminal — enable memory (pick one)
export CORTEX_ENABLE_MEMORY=1        # environment variable
# or set in ~/.snowflake/cortex/settings.json: "enableMemory": true
```

remember — save something CoCo should know in every future session:

```
cortex memory remember "use conventional commits: feat:, fix:, docs:, chore:, refactor:, test:"
cortex memory remember "production db is read-only, use role PROD_READ for queries"
cortex memory remember "default warehouse is COMPUTE_WH_M, only use XL for full-table scans"
cortex memory remember "prefer async/await over .then() in this codebase"
```

recall — find a specific memory by topic:

```
cortex memory recall "commit format"
cortex memory recall "production access"
cortex memory recall "warehouse sizing"
```

list / drop — manage stored memories:

```
cortex memory list              # show all memories with IDs
cortex memory drop <id>         # remove a specific memory by ID
```

> [!TIP] Natural language works too:
>
> - Say “remember that we deploy to us-east-1” and CoCo stores it.
> - Say “what’s our commit convention?” and CoCo recalls it from memory without a command.

Environment Variables

Variable	Effect
CORTEX_ENABLE_MEMORY	Enable cross-session memory

Variable	Effect
	(alternative to settings)
COCO_DISABLE_CRON	Disable /loop and all scheduling tools
COCO_DISABLE_ROUTINES	Disable routines
CORTEX_CODE_STREAMING	Enable code streaming
CORTEX_AGENT_USE_LOCAL_ORCHESTRATOR	Use local orchestrator (developer/ debug mode)
CTX_STEP_ENFORCEMENT	Enforce ctx step tracking discipline
CORTEX_BROWSER_HEADLESS	Run browser automation without a visible window
CORTEX_CODE_ENABLE_SNOWFLAKE_MANAGED_MCP_SERVERS	Enable Snowflake-managed MCP servers
CORTEX_DISABLE_TODO_TOOL	Disable the todo/task tool

Set inline for a single session:

```
# run in terminal
CORTEX_ENABLE_MEMORY=1 CORTEX_BROWSER_HEADLESS=1 cortex
```

Settings Worth Knowing

Configure via `/settings` or edit `~/.snowflake/cortex/settings.json`.

Key	Default	What it does
agentMode	standard	`standard` = balanced; `code` = optimized for code-heavy tasks (higher tool call budget)

Key	Default	What it does
autoAcceptPlans	false	Skip the “approve plan?” prompt
enableMemory	false	Enable cross-session memory
tgrepEnabled	true	Enable semantic code search
sqlDefaultTimeoutSeconds	180	Max wait for SQL queries
diffDisplayMode	unified	unified (git-style) or side_by_side
bashDefaultTimeoutMs	180000	Timeout for shell commands
mcpWait	false	Wait for all MCP servers before starting (useful in CI)
disableCron	false	Disable /loop scheduling
autoUpdate	true	false to disable automatic version updates

Minimal ~/.snowflake/cortex/settings.json to start from:

```json
{
  "enableMemory": true,
  "autoAcceptPlans": false,
  "autoUpdate": false,
  "sqlDefaultTimeoutSeconds": 300
}
```

CLI Subcommands

Run outside the interactive session — useful in scripts, CI, and automation.

Command	What it does
cortex search object "<query>"	Find tables, views, schemas by name or description
cortex search table-details "DB.SCHEMA.TABLE"	Full column metadata for a table
cortex search docs "<query>"	Search Snowflake product docs
	Search Marketplace listings

Command	What it does
cortex search marketplace "<query>"	
cortex lineage DB.SCHEMA.TABLE --direction both --tree	Trace upstream and downstream lineage
cortex connections list	List available connections
cortex connections set <name>	Switch active connection
cortex semantic-views list	List semantic views in the account
cortex worktree create <branch>	Create a git worktree (also available via /worktree)
cortex skill list	List installed skills
cortex memory remember "<text>"	Store a memory (requires enableMemory: true)
cortex update	Update to the latest version

For the full command list: CLI Reference

MCP Integration

Config file: ~/.snowflake/cortex/mcp.json | Manage: cortex mcp add | list | remove | In-session: /mcp

Tool naming inside CoCo: mcp__<server>__<tool> (e.g. mcp__github__create_pull_request)

Supported transports: http, sse, stdio

Add a server (example: GitHub):

```
# run in terminal
cortex mcp add
# ↳ transport=stdio, command=npx, args=-y @modelcontextprotocol/
server-github
# ↳ env: GITHUB_PERSONAL_ACCESS_TOKEN=<your-token>
```

Other popular servers (all via npx -y @modelcontextprotocol/server-<name>): filesystem, postgres, brave-search, slack

Set mcpWait: true in settings if MCP tools must be ready before the first prompt (CI/automation).

```
# run in terminal
cortex mcp list                    # show configured servers
cortex mcp remove <server_name>    # remove a server
```

For setup details: MCP docs

Hooks

Hooks fire shell scripts in response to lifecycle events. Configure in `~/.snowflake/cortex/hooks/`.

Event	When it fires
`UserPromptSubmit`	User submits a prompt
`PreToolUse`	Before any tool call
`PostToolUse`	After any tool call
`PermissionRequest`	When a permission prompt appears
`Stop`	Agent stops
`SessionStart`	Session begins
`SessionEnd`	Session ends
`PreCompact`	Before conversation compaction

View and test configured hooks:

```
/hooks
```

Tips & Gotchas

- **`/plan` before anything risky.** CoCo reads the codebase, drafts a plan, and waits for your approval. For multi-file changes this is the difference between a clean refactor and a mess.
- **`@file` does NOT inject content — `@{file}` does.** `@src/auth.py review this` just references the path; CoCo may or may not read it. `@{src/auth.py} review this` injects the full contents every time. If CoCo seems to "not know" a file, switch to `@{`.
- **`#TABLE` loads schema automatically.** Type `#` then the table name — CoCo fetches column definitions before writing any SQL. No need to describe the table.
- **`/bg` for long jobs.** Fire off a background agent to run a big refactor or analysis while you keep chatting. Results arrive as a notification.
- **`cortex search docs` is semantic.** "how do I set target lag for dynamic tables" finds the right page faster than a Google search.
- **Secrets stay out of chat.** Use `/secrets` to store API keys and tokens. CoCo injects them at runtime without exposing them in the session transcript.

References

- Cortex Code CLI
- CLI Reference (slash commands, batch mode)
- Keyboard Shortcuts

- Bundled Skills
- Settings Reference
- Extensibility (hooks, plugins, MCP)
- Changelog